1. Nazwa klas i metod

1.1 **Nazwy klas**: Używaj notacji CamelCase (np. Chess, Pawn, King).
1.2 **Nazwy metod**: Używaj notacji snake_case (np. handle_square_click, move_piece).

---

2. Komentarze

2.1 **Komentowanie kodu**: Komentuj kod w sposób zrozumiały i klarowny, wyjaśniając, co robi każda funkcja, metoda lub fragment kodu.
2.2 **Język komentarzy**: Używaj języka angielskiego w komentarzach, aby kod był zrozumiały dla szerszej grupy programistów.
2.3 **Dokumentowanie złożonych funkcji**: Zawsze dodawaj komentarze wyjaśniające logikę szczególnie złożonych funkcji i metod.

---

3. Formatowanie kodu

3.1 **Wcięcia**: Używaj tabulatorów do wcięć w kodzie.
3.2 **Puste linie**: Stosuj puste linie między metodami i klasami, aby poprawić czytelność kodu i oddzielić różne sekcje w kodzie.

---

4. Struktura kodu

4.1 **Podział na mniejsze fragmenty**: Jeśli dana funkcja lub metoda staje się zbyt długa, rozważ podzielenie jej na mniejsze, bardziej zrozumiałe fragmenty.
4.2 **Zasada pojedynczej odpowiedzialności**: Każda funkcja/metoda powinna odpowiadać tylko za jedną konkretną rzecz, co ułatwia jej zrozumienie i testowanie.

---

5. Nazwy zmiennych

5.1 **Jasne nazwy**: Używaj nazw zmiennych, które jednoznacznie opisują ich przeznaczenie. Zmienna powinna mieć pełny opis, np. start_pos zamiast pos.

5.2 **Unikanie skrótów**: Unikaj skrótów, które mogą być trudne do zrozumienia w kontekście kodu. Na przykład piece_color jest lepsze niż p_col.

---

6. Unikanie magicznych liczb

6.1 **Stałe zamiast magicznych liczb**: Unikaj tzw. magicznych liczb (np. 8, 2, 3), które mogą być trudne do zrozumienia.
6.2 **Zamiast magicznych liczb wprowadź stałe**: Zamiast magicznych liczb, wprowadź stałe z odpowiednimi nazwami, np. BOARD_SIZE = 8, MAX_PIECES = 16.

---

7. Zarządzanie błędami

7.1 **Obsługa błędów**: Dodaj odpowiednie mechanizmy obsługi błędów i wyjątków tam, gdzie może dojść do problemów.

.

Przykładowy fragment kodu:

```python
from nicegui import ui
from functools import partial
from src.ui.components.chess_board import ChessBoard
from src.ui.components.chess_castling import ChessCastling
from src.ui.components.chess_endgame import ChessEndgame
from src.ui.components.chess_pieces import King, Pawn

class Chess:  2 usages  Zaichenko Illia +1
    def __init__(self):  Zaichenko Illia +1
        # Initialize board size and set the initial state
        self.board_row = 8
        self.board_col = 8
        self.board_class = ChessBoard(self)
        self.end_game_class = ChessEndgame(self)
        self.handling_castling = ChessCastling(self)
        self.start_pos = None
        self.turn = 'white'
        self.grid = None
        self.board = self.board_class.init_stalemate()
        self.squares = [[None for _ in range(self.board_col)] for _ in range(self.board_row)]
        self.white_captured = []
        self.black_captured = []
        self.last_move = None
        self.create_chessboard()

    def create_chessboard(self):  1 usage  Zaichenko Illia +1
        self.grid = ui.grid(columns=8).classes('w-auto m-auto gap-0')
        with self.grid:
            for i in range(self.board_row):
                for j in range(self.board_col):
                    color = '#769656' if (i + j) % 2 == 0 else '#EEEED2'

                    # Make the background div non-interactive
                    with ui.element('div').style(f'background-color: {color}; width: 100px; height: 100px;').on(
                            'click', partial(self.handle_square_click,  *args: i, j)) as square:
                        self.squares[i][j] = square
                        piece = self.board[i][j]
                        if piece:
                            piece_type = piece.__class__.__name__.lower()
                            piece_color = piece.color
                            image_path = f'./src/img/{piece_color}-pieces/{piece_type}-{piece_color}.png'

                            # Make the chess pieces interactive
                            with square:
                                ui.image(image_path).style('pointer-events: none;')
                                # Allow clicks on the piece image
                                if isinstance(piece, King):
                                    self.add_king_context_menu(square, piece)
        self.update_captured_pieces()
```